



Python Programming Fundamentals

Driver and Main Function

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
7. Printing Product Details



Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
7. Printing Product Details



1.1 Separating Concerns

In OOP, we separate:

- **Model classes** — Product, Shop (define data + behaviour)
- **Driver** — the file that **uses** those classes (the “main” program)

Benefits:

- Classes stay reusable across multiple programs
- Driver is easy to read — it tells the story of the program
- Easier to test each part independently



Outline

1. What is a Driver?
- 2. The `main()` Function**
3. Importing the Class
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
7. Printing Product Details



2.1 Writing a clean entry point

```
# driver.py
from product import Product

def main():
    apple = Product("Apple", 1.99, 101)
    bread = Product("Bread", 2.50, 102)
    milk = Product("Milk", 1.20, 103)

    print(apple)
    print(bread)
    print(milk)

# Update a product
apple.set_price(2.29)
print(f"Updated price: {apple}")
```



2.1 Writing a clean entry point

```
if __name__ == "__main__":  
    main()
```



Outline

1. What is a Driver?
2. The `main()` Function
- 3. Importing the Class**
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
7. Printing Product Details



3.1 Using from ... import

```
from product import Product
```

- product is the **file name** (without .py)
- Product is the **class name** inside that file
- Both files must be in the **same folder**

File structure:

```
shop_app/  
├── product.py    ← defines the Product class  
└── driver.py     ← imports and uses Product
```



Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
- 4. Creating Products in `main()`**
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
7. Printing Product Details



4. Creating Products in main()

```
def main():  
    # Create several Product instances  
    products = [  
        Product("Apple", 1.99, 101),  
        Product("Bread", 2.50, 102),  
        Product("Milk", 1.20, 103),  
        Product("Butter", 3.00, 104),  
    ]  
  
    # Print each product  
    for p in products:  
        print(p)  
  
    print(f"\nTotal products: {len(products)}")
```

Output:



4. Creating Products in `main()`

```
Product [101]: Apple – €1.99  
Product [102]: Bread – €2.50  
...
```



Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
4. Creating Products in `main()`
- 5. The `if __name__ == "__main__"` Guard**
6. Calling Methods on Objects
7. Printing Product Details



5.1 Why do we need it?

```
if __name__ == "__main__":  
    main()
```

- When Python runs a file directly, `__name__` equals `"__main__"`
- When a file is **imported** by another module, `__name__` equals the module name
- The guard ensures `main()` only runs when the file is run **directly**

Without the guard: importing `driver.py` elsewhere would run `main()` immediately — almost always unwanted.



Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
- 6. Calling Methods on Objects**
7. Printing Product Details



6. Calling Methods on Objects

```
def main():  
    apple = Product("Apple", 1.99, 101)  
  
    # Read attribute  
    print(apple.get_name())      # Apple  
    print(apple.get_price())     # 1.99  
    print(apple.get_product_id()) # 101  
  
    # Update attribute  
    apple.set_price(2.49)  
    apple.set_name("Granny Smith Apple")  
  
    print(apple)  
    # Product [101]: Granny Smith Apple – €2.49
```




Outline

1. What is a Driver?
2. The `main()` Function
3. Importing the Class
4. Creating Products in `main()`
5. The `if __name__ == "__main__"` Guard
6. Calling Methods on Objects
- 7. Printing Product Details**



7.1 Formatted output

```
def print_product_details(product: Product):  
    print("=" * 40)  
    print(f"    ID      : {product.get_product_id()}")  
    print(f"    Name   : {product.get_name()}")  
    print(f"    Price  : €{product.get_price():.2f}")  
    print("=" * 40)  
  
def main():  
    apple = Product("Apple", 1.99, 101)  
    print_product_details(apple)
```

Output:

```
=====
```

ID	:	101
Name	:	Apple



7.1 Formatted output

Price : €1.99

=====



Thanks for Watching - Any questions?